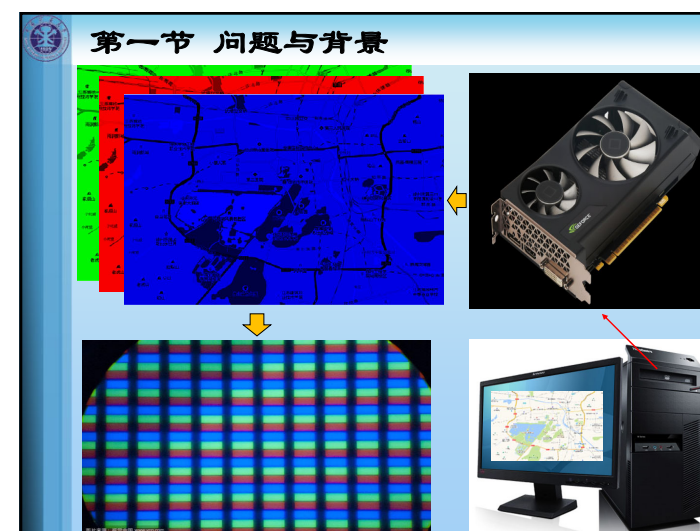
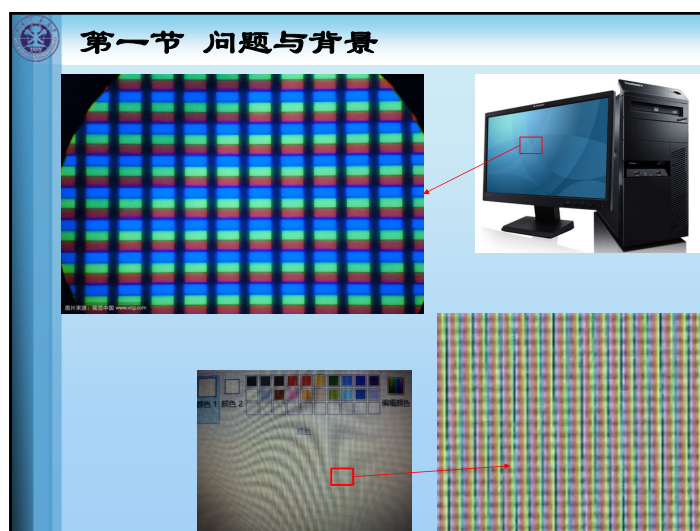
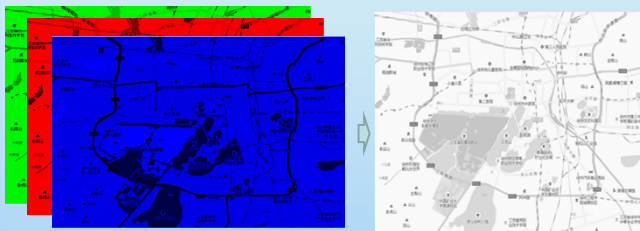




第一节 问题与背景



- 1) 确定图形的像素;
- 2) 确定图形的颜色;

第一节 问题与背景

数字地图

地图数据库

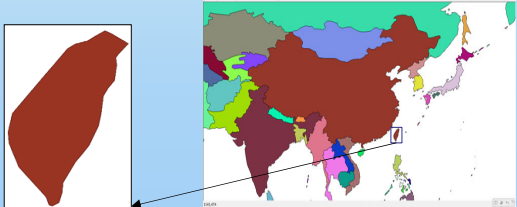
如何显示?



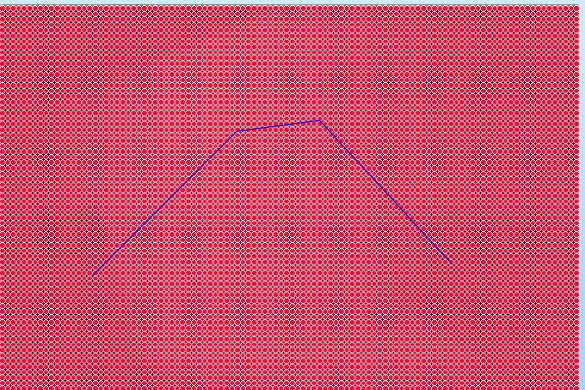
电子地图

第一节 问题与背景

- ✓线如何绘制? (第三章)
- ✓面如何绘制? (第四章)
- ✓图形如何变换与裁剪? (第五章)
- ✓地图符号如何绘制? (第六章)

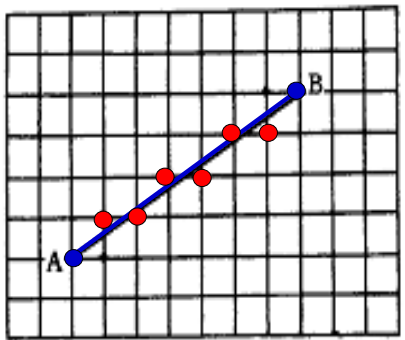


第一节 问题与背景



再进行进一步抽象

第一节 问题与背景



- ✓ 点阵表示
用像素点色表示图形几何形状
- ✓ 矢量表示
用点坐标来表示图形的几何形状

问题的本质：如何将矢量表示的直线转换为点阵表示的直线？

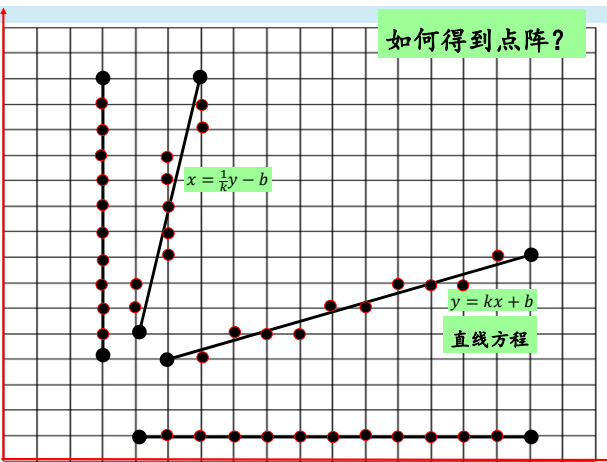
第一节 问题与背景

- 如何得到点阵？
- 如何更快地得到点阵？
- 如何得到质量更好的点阵？

从图形对象的几何信息出发，确定最佳逼近图形的象素集合，并用指定的颜色和灰度设置象素的过程称为**图形的扫描转换或光栅化**。

第二节 直线生成算法

如何得到点阵？



直线方程

第二节 直线生成算法

1 直线方程法

化繁为简，逐个突破！

- 1) 求出直线方程 $y=kx+b$
- 2) 求出 k 及 b
- 3) If $|k| \leq 1$, 代入整数 x 得到 y
- 4) If $|k| > 1$, 代入整数 y 得到 x

存在哪些瓶颈？

$y = kx + b$

$+ > - > \times > \div$
整数运算 > 浮点运算

精益求精：还能更快吗？

第二节 直线生成算法

2 数值微分法 (DDA) (1/5)

➤ $|k| \leq 1$ 的直线方程

$$Y = kX + b$$

$$k = (Y_1 - Y_0) / (X_1 - X_0)$$

$$Y_{i+1} = kX_{i+1} + b$$

$$= k(X_i + Dx) + b$$

$$= kX_i + b + kDx$$

$$= Y_i + kDx$$

令 $Dx = 1$, 则 $Y_{i+1} = Y_i + k$

第二节 直线生成算法

2 数值微分法 (DDA) (2/5)

➤ $|k| > 1$ 的直线方程

$$x = k'y + b'$$

$$k' = (X_1 - X_0) / (Y_1 - Y_0)$$

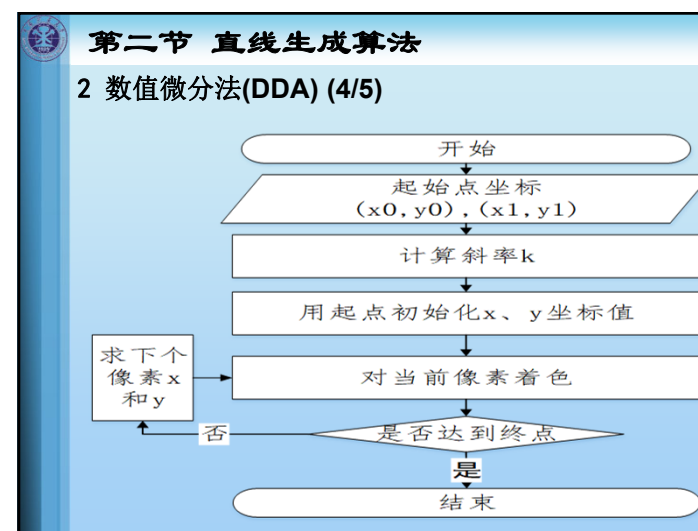
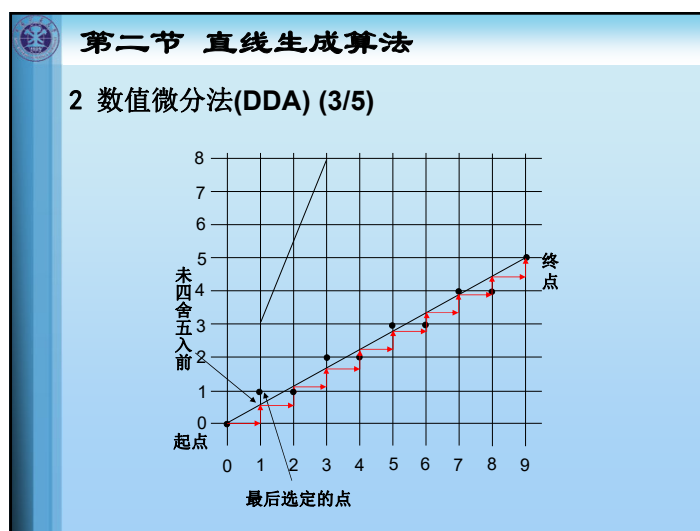
$$X_{i+1} = k'Y_{i+1} + b'$$

$$= k'(Y_i + Dy) + b'$$

$$= k'Y_i + b' + k'Dy$$

$$= X_i + k'Dy$$

令 $Dy = 1$, 则 $X_{i+1} = X_i + k'$



第二节 直线生成算法

2 数值微分法(DDA) (5/5)

```
void Cdda1View::DDA(int x0, int y0, int x1, int y1, long color, CDC* pDC)
{
    float k=(float)(y1-y0)/(x1-x0);
    float y=y0;
    for(int x=x0;x<=x1;x++)
    {
        pDC->SetPixel(x,int(y+0.5),color);
        y+=k;
    }
}
```

换一下起,点终点顺序呢?

问题分析:
存在浮点运算、取整运算、不利于硬件实现。

精益求精: 还能更快吗?

第二节 直线生成算法

- ✓ DDA算法的启发

直线生成问题可转换为下一个像素相对于当前像素的走向问题。
- ✓ 改进的关键: 加速下一个像素的计算。
- ✓ 中点画线算法
- ✓ Bresenham算法

第二节 直线生成算法

3 中点画线法 (1/7)

$0 < k < 1$

✓ M 在 Q 的下方
 P_2 为下一像素点

✓ M 在 Q 的上方
 P_1 为下一像素点

中点 $M(Xp+1, Yp+0.5)$

$P(Xp, Yp)$ 当前像素点

$P1(Xp+1, Yp)$

$P2(Xp+1, Yp+1)$

第二节 直线生成算法

3 中点画线法 (2/7)

直线段方程为

$$F(x, y) = ax + by + c = 0$$

M 代入方程, 判断符号。

$a = y_0 - y_1, b = x_1 - x_0$

$d = F(M) = F(x_p + 1, y_p + 0.5)$
 $= a(x_p + 1) + b(y_p + 0.5) + c$

$d < 0$, 取 P_2 (右上)
 $d > 0$, 取 P_1 (正右)
 $d = 0$, P_1, P_2 均可, 约定取 P_1 ;

✓ M 在 Q 的下方
 P_2 为下一像素点

✓ M 在 Q 的上方
 P_1 为下一像素点

$0 < k < 1$

$P2(Xp+1, Yp+1)$

中点 $M(Xp+1, Yp+0.5)$

$P(Xp, Yp)$ 当前像素点

$P1(Xp+1, Yp)$

第二节 直线生成算法

3 中点画线法 (3/7)

1) $d < 0$, 取右上像素 P_2
 下下个像素位置?
 $d_{i+1} = F(x_p+2, y_p+1.5)$
 $= a(x_p+2) + b(y_p+1.5) + c$
 $= d_i + a + b$ (增量为 $a+b$)。

2) $d \geq 0$, 则正右像素 P_1
 下下个像素位置?
 $d_{i+1} = F(x_p+2, y_p+0.5)$
 $= a(x_p+2) + b(y_p+0.5) + c$
 $= d_i + a$ (增量为 a)

$d = F(x_p+1, y_p+0.5)$
 $d < 0$, 取 P_2 (右上)
 $d > 0$, 取 P_1 (正右)
 $d = 0$, 取 P_1 (正右)

$0 < k < 1$

第二节 直线生成算法

3 中点画线法 (4/7)

$d_i < 0$, 取右上, $d_{i+1} = d_i + a + b$
 $d_i > 0$, 取正右, $d_{i+1} = d_i + a$

画线从 (x_p, y_p) 开始, d 的初值:
 $d_0 = F(x_p+1, y_p+0.5)$
 $= F(x_p, y_p) + a + 0.5b$
 因 $F(x_p, y_p) = 0$, 所以

$d_0 = a + 0.5b$
 $a = y_0 - y_1, b = x_1 - x_0$

$D_i < 0$, 取右上, $D_{i+1} = D_i + 2a + 2b$
 $D_i > 0$, 取正右, $D_{i+1} = D_i + 2a$

精益求精: 还能让他更快吗?
 既然像素取舍只与 d_i 的符号有关, 那么只需要判断与 0 的关系即可。对 d_i 同乘 2。

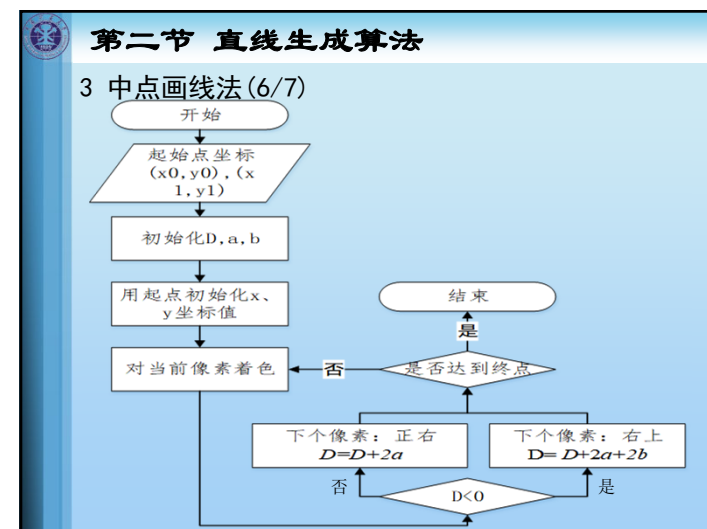
$D_0 = 2d_0 = 2a + b$

第二节 直线生成算法

$a = y_0 - y_1, b = x_1 - x_0, D_0 = 2d_0 = 2a + b$
 $D_i < 0$ (右上): $D_{i+1} = D_i + 2a + 2b$
 $D_i > 0$ (正右): $D_{i+1} = D_i + 2a$

$a = -4; b = 7; D_0 = -1;$
 $2a + 2b = 6;$
 $2a = -8$

Step	X	Y	D
1	0	0	-1
2	1	1	5
3	2	1	-3
4	3	2	3
5	4	2	-5
6	5	3	1
7	6	3	-7
8	7	4	-1



第二节 直线生成算法

3 中点画线法 (7/7)

```

void MidpointLine (int x0,int y0,int x1,int y1,int color)
{
    int a,b,d1,d2,d,x,y;
    a=y0-y1;b=x1-x0;
    d1=2*a; d2=2*(a+b);
    x=x0;y=y0;
    d=2*a+b;
    while (x<=x1)
    {
        SetPixel(x,y,color);
        if(d<0)
            {x++;y++;d+=d2;}
        else
            {x++;d+=d1;}
    }
}

```

$0 < k < 1$

第二节 直线生成算法

4 Bresenham (1/4)

第二节 直线生成算法

4 Bresenham (2/4)

$0 < k < 1$

$x_i = x_{i-1} + 1$

$y_i = y_{i-1} + k$

设误差项: $d_i = y_i - y_{i-1,r} - 0.5$

$d_i \geq 0$, 右上, $y_i = y_{i-1,r} + 1$

$d_i < 0$, 正右, $y_i = y_{i-1}$

第二节 直线生成算法

4 Bresenham (3/4)

$d_i \geq 0$, 右上

$y_{i,r} = y_{i-1,r} + 1$

$d_i < 0$, 正右

$y_{i,r} = y_{i-1,r}$

因为 $y_i = y_{i-1} + k$

$d_i = y_{i-1} + k - y_{i-1,r} - 0.5$

$d_i = \begin{cases} y_{i-1} - (y_{i-2,r} + 1) + k - 0.5 & d_{i-1} \geq 0 \\ y_{i-1} - y_{i-2,r} + k - 0.5 & d_{i-1} < 0 \end{cases}$

$d_i = \begin{cases} d_{i-1} + k - 1 & d_{i-1} \geq 0 \\ d_{i-1} + k & d_{i-1} < 0 \end{cases}$

$d_{i+1} = \begin{cases} d_i + k - 1 & d_i \geq 0 \\ d_i + k & d_i < 0 \end{cases}$

第二节 直线生成算法

4 Bresenham (4/4)

$d_i \geq 0$, 取右上, $d_{i+1} = d_i + k - 1$

$d_i < 0$, 取正右, $d_{i+1} = d_i + k$

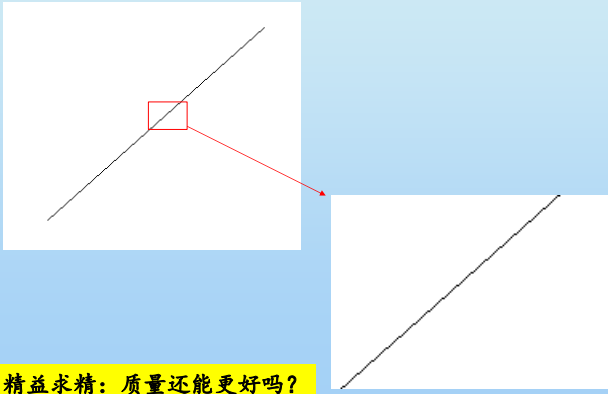
$d_0 = -0.5$ ($y_0 = y_{0,r}$)

$k = \frac{\Delta y}{\Delta x}$

精益求精: 还能让他更快吗?

第二节 直线生成算法

5、提高直线生成的质量

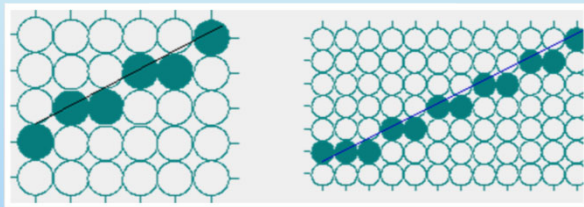


精益求精: 质量还能更好吗?

第二节 直线生成算法

5、提高直线生成的质量

(1) 提高分辨率



代价: 以4倍的存储器空间和扫描转换时间

虽简单, 但不经济, 且也只能减轻而不能消除锯齿问题。

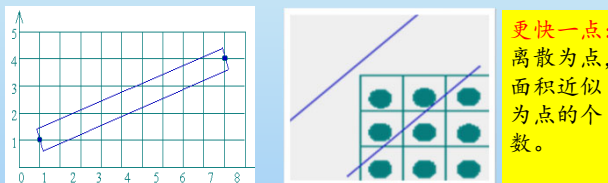
精益求精: 还能更好的解决方法吗?

第二节 直线生成算法

5、提高直线生成的质量

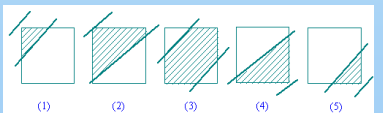
(2) 区域采样

将直线段看作具有一定宽度的狭长矩形。根据相交区域面积的大小确定该像素的亮度值。



更快一点: 离散为点, 面积近似为点的个数。

面积比确定灰度值



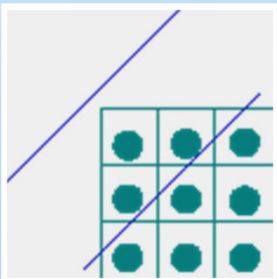
第二节 直线生成算法

5、提高直线生成的质量

做得更好一些!

(3) 加权区域取样

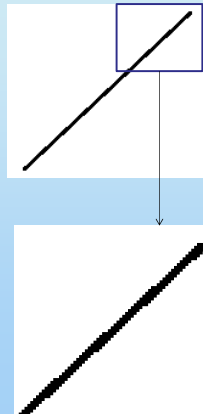
相交区域对象素亮度的贡献依赖于该区域与象素中心的距离。



$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

第二节 直线生成算法

- ✓ 离散量表示连续量引起的失真现象称为**走样**。
- ✓ 用以减少或消除失真现象的技术称为**反走样**。
- ✓ 反走样技术
 - 提高分辨率、区域采样、加权区域采样



作业一：中点画线算法完善与补充

一、作业内容

结合图形，推导任意斜率中点画线算法的递推公式，及对应下一像素的取法。

在此基础上，画出任意斜率中点画线算法的流程图。

二、作业要求

- 1) 第2次上机实验前完成；
- 2) 在第2次上机实验时携带纸质作业；
- 3) 将本次作业内容作为第2次上机实验报告的一部分。

实验二：中点画线算法直线绘制

一、实验目的

- 1) 了解中点画线算法的基本原理；
- 2) 学会使用VC++实现直线生成算法编程。

二、实验内容

对于任意输入的两点（斜率任意、起始点顺序任意），利用中点画线算法实现的直线生成与绘制。

三、实验要求

- 1) 建立的工程名以“班级_学号_实验序号”命名；
- 2) 撰写并提交实验报告，内容包括：封面、实验目的、实验内容、算法推导、算法流程图、**关键代码**、执行结果、实验体会。

实验二：中点画线算法直线绘制

3) 要求能正确绘制以下6条直线。

直线	x0	y0	x1	y1
1	200	500	800	500
2	500	200	500	800
3	800	650	200	350
4	650	800	350	200
5	200	650	800	350
6	350	800	650	200

第三节 圆的生成算法

第三节 圆的生成算法

1. 圆弧扫描算法

$$X^2 + Y^2 = R^2$$

$$Y = \pm\sqrt{R^2 - X^2}$$

在一定范围内，每给定一X值，可得一Y值。

缺点：浮点运算，开方，像素不均匀。

还能更好的解决方法吗？

第三节 圆的生成算法

2. 角度DDA法 (1/2)

圆的参数方程： $x = x_0 + R \cdot \cos\theta$
 $y = y_0 + R \cdot \sin\theta$

求导： $dx = -R \cdot \sin\theta d\theta$
 $dy = R \cdot \cos\theta d\theta$

因此：

$$x_{n+1} = x_n + dx = x_n - (y_n - y_0)d\theta$$

$$y_{n+1} = y_n + dy = y_n + (x_n - x_0)d\theta$$

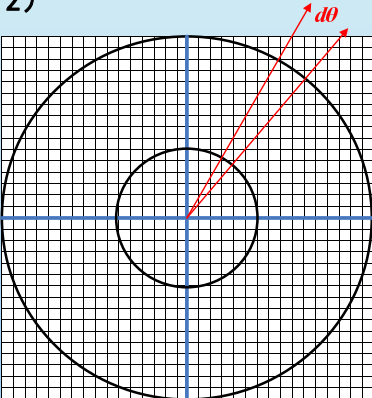
确定x,y的初值及dθ值后，即可以增量方式获得圆周上的坐标，然后取整可得像素坐标。

第三节 圆的生成算法

2. 角度DDA法 (2/2)

缺点:

- ✓ 采用浮点运算、乘法运算
- ✓ $d\theta$ 的取值不容易确定, 与半径大小有关

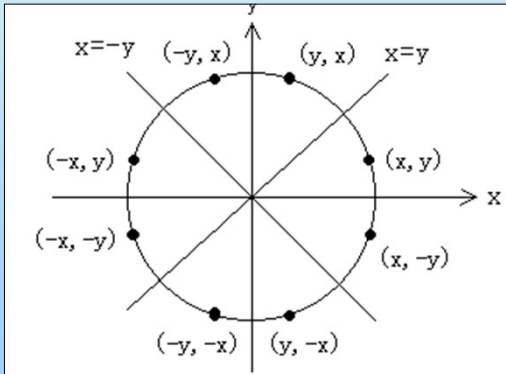


还能更好的解决方法吗?

第三节 圆的生成算法

3. 中点法 (1/5)

利用圆的对称性, 只须讨论1/8圆。



第三节 圆的生成算法

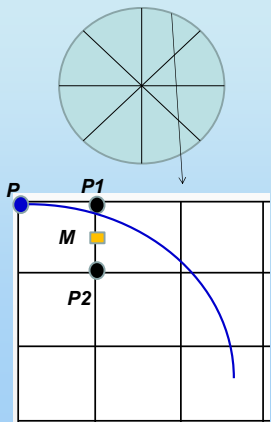
3. 中点法 (2/5)

圆弧的隐函数:
 $F(X, Y) = X^2 + Y^2 - R^2 = 0$

对于中点 $M(Xp+1, Yp-0.5)$:

当 $F(M) < 0$ 时, M 在圆内, $P1$ 距离圆弧更近, 取 $P1$;

当 $F(M) > 0$ 时, 取 $P2$



第三节 圆的生成算法

3. 中点法 (3/5)

$$d = F(M) = F(Xp+1, Yp-0.5)$$

$$= (Xp+1)^2 + (Yp-0.5)^2 - R^2$$

$d_i < 0$, 取 $P1$ (正右)

$$d_{i+1} = F(Xp+2, Yp-0.5)$$

$$= d_i + 2Xp + 3$$

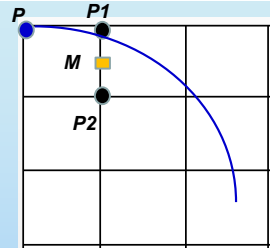
$d_i \geq 0$, 取 $P2$ (右下)

$$d_{i+1} = F(Xp+2, Yp-1.5)$$

$$= d_i + (2Xp + 3) + (-2Yp + 2)$$

$$= d_i + 2(Xp - Yp) + 5$$

d 的初始值, $M(1, R-0.5)$
 $d_0 = F(1, R-0.5)$
 $= 1.25 - R$



第三节 圆的生成算法

3. 中点法 (4/5)

MidpointCircle(int r) 有浮点数

```

{
    int x,y;
    float d;
    x=0; y=r; d=1.25-r;
    while(x<y)
    {
        setpixel(x,y);
        if(d<0)
        {
            d+=2*x+3; x++;
        }
        else
        {
            d+=2*(x-y)+5;
            x++;y--;
        }
    }
}

```

第三节 圆的生成算法

3. 中点法 (5/5)

令: $e=d-0.25$

$d_0=1.25-R$	$e_0=1-R$
$d_i < 0$, 取P1(正右) $d_{i+1}=d_i+2Xp+3$	$e_i < -0.25$, 取P1(正右) $e_{i+1}=e_i+2Xp+3$
$d_i \geq 0$, 取P2(右下) $d_{i+1}=d_i+2(Xp-Yp)+5$	$e_i \geq -0.25$, 取P2(右下) $d_{i+1}=d_i+2(Xp-Yp)+5$

由于 e 的初值为整数, 且在运算过程中的增量也是整数, 故 e 始终是整数, 所以 $e < -0.25$ 等价于 $e < 0$

第三节 圆的生成算法

4. Bresenham算法 (1/3)

当 $x_{hi}^2 + y_{hi}^2 - R^2 \geq R^2 - (x_{li}^2 + y_{li}^2)$ 时, 应该取 L_i , 否则取 H_i 。

令 $d_i = x_{hi}^2 + y_{hi}^2 + x_{li}^2 + y_{li}^2 - 2R^2$, 当 $d_i \geq 0$ 时应该取 L_i , 否则取 H_i 。

第三节 圆的生成算法

4. Bresenham算法 (2/3)

$d_i = x_{hi}^2 + y_{hi}^2 + x_{li}^2 + y_{li}^2 - 2R^2$

$d_i < 0$, 取 H_i (正右), 即 $y_{i+1}=y_i$, $d_{i+1}=d_i+4x_i+6$;
$d_i \geq 0$, 取 L_i (右下), 即 $y_i=y_i-1$, $d_{i+1}=d_i+4(x_i-y_i)+10$

易知: $x_0=0$; $y_0=R$; $d_0=3-2R$;

第三节 圆的生成算法

4. Bresenham算法 (3/3)

```

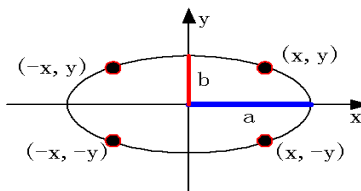
BresenhamCircle(int r, int color)
{
    int x=0,y=r,d=3-2*r;
    while(x<=y)
    {
        circlepoints(x,y,color);
        if (d<0)
            d=d+4*x+6;
        else
        {
            d=d+4*(x-y)+10;
            y--;
        }
        x++;
    }
}

```

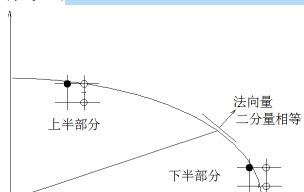
$d_i < 0$, 取 H_i (正右), 即 $y_{i+1} = y_i$,
 $d_{i+1} = d_i + 4x_i + 6$;
 $d_i \geq 0$, 取 L_i (右下), 即 $y_{i+1} = y_i - 1$,
 $d_{i+1} = d_i + 4(x_i - y_i) + 10$
 易知: $x_0 = 0$; $y_0 = R$; $d_0 = 3 - 2R$;

第三节 圆的生成算法

5. 椭圆的生成



长半轴为a, 短半轴为b



第四节 字符、线型与线宽

1. 字符生成技术

- 字符

是一种图形, 通常指数字、字母、汉字等符号。计算机中字符由一个数字编码唯一标识。

ASCII: 美国标准信息交换码

GB2312/GBK/Big5/JIS, UNICODE/UTF-8
- 字库

为了在显示器等输出设备上输出字符, 系统中必须装备有相应的字库。字库存储了每个字符的形状信息, 分为矢量和点阵型两种。

第四节 字符、线型与线宽

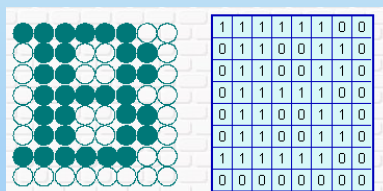
1. 字符生成技术

- ✓ 点阵字符

点阵字符库中, 每个字符由一个位图表示, “1”表示对应像素置为字符颜色, “0”表示对应像素置为背景颜色。

通常采用16×24的位图来表示, 一个汉字需要384位, 共需6763×384≈324kB

实际中, 有多种字体, 每种字体又有多种型号, 占用较大存储空间。



(a) 点阵字符 (b) 点阵字符库中的位图表

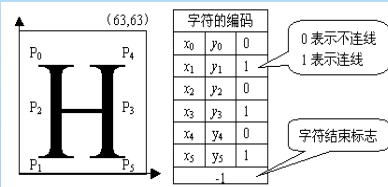
第四节 字符、线型与线宽

2、字符生成技术

• 矢量字符

记录字符的笔画信息而不是整个位图。一个字
符由若干笔画构成，每一笔画由两 endpoint 确定，对于
每一个 endpoint，只需保存坐标值及与前一点的连接关
系即可。

具有存储空间小
、美观、变换方
便等特点。



字符的编码		
x_0	y_0	0
x_1	y_1	1
x_2	y_2	0
x_3	y_3	1
x_4	y_4	0
x_5	y_5	1
		-1

0 表示不连线
1 表示连线

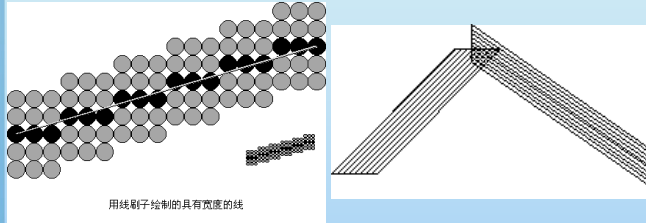
字符结束标志

图 3-90 矢量字符的表示

第四节 字符、线型与线宽

2、线宽生成技术

(1) 线刷子方法



用线刷子绘制的具有宽度的线

优点：算法简单、效率高。

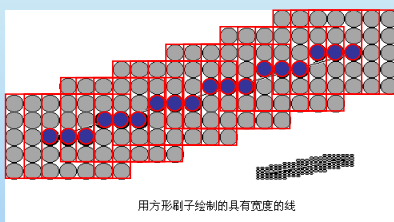
缺点：1) 线的始末端总是水平或垂直的。当线宽较大时，看起来很不自然。当比较接近水平的线与比较接近垂直的线汇合时，汇合处外角将有缺口。

2) 水平线或垂直线，最粗。45°斜线，最细， $1/\sqrt{2} \approx 0.7$ 倍。

第四节 字符、线型与线宽

2、线宽生成技术

(2) 方形刷子法



用方形刷子绘制的具有宽度的线

实现方法：把方形中心对准单像素宽的线条上各个像素，并把方形内的像素全部置成线条颜色。

始末端也是水平或垂直的，且线宽与线条方向有关。
水平线与垂直线，线宽最小，而对于斜率为 ± 1 的线条，线宽最大，为垂直（水平）的 $\sqrt{2}$ 倍。

缺点：重复地写像素。

第四节 字符、线型与线宽

3、线型生成技术

线型通过像素模板定义，像素模板则由数字
“0”和“1”组成，不同的模板表示不同的线型。

例如，11111111110000000000111111111100

画线时，根据模板上的数字，将沿线段路径上的
像素置为前景或背景颜色。

```
if(串[i%32]) setPixel(x,y,color);
```



$F(x,y)>0$, 点不一定在直线上方?

假定 k 存在, 直线方程可表示为:

$$y=kx+t \quad (1)$$

将式1向左移, 则得到

$$G(x,y)=y-kx-t=0 \quad (2)$$

如果 $G(x,y)>0$, 则 $y>kx+t$, 那么 y 值大于 x 值所对应直线上点的纵坐标值, 即点 (x,y) 在直线上方。

根据两已知点坐标, 可求得 $k=(y_1-y_0)/(x_1-x_0)$, $c=y_0-kx_0$

对式2 两边乘以一个正数 (x_1-x_0) , 可得:

$$\begin{aligned} (x_1-x_0) G(x,y) &= (x_1-x_0) y - (x_1-x_0) k x + (x_1-x_0) t \\ &= (y_1-y_0)x + (x_1-x_0) y + (x_1-x_0) t \end{aligned}$$

令 $F(x,y) = (x_1-x_0) G(x,y) = ax+by+c$

则: $a=y_1-y_0$, $b=x_1-x_0$

显然, $F(x,y)>0$ 等价于 $G(x,y)>0$ 。

为此, $F(x,y)>0$ 时, 点 (x,y) 在直线上方